B.Tech Computer Science and Engineering (AI & ML) Academic Mini Project

RoadVision

Intelligent Road Monitoring System

Product Requirements Document (PRD)

Requirements for a locally operated, computer-vision-based road-damage detection application supporting image, recorded-video and live-webcam input.

Document version	1.1 (revised after requirements review; supersedes 1.0)
Project type	B.Tech CSE (AI & ML) Mini Project
Team size	2 (Ram, Shakib)
Development timeline	7 days
Target platform	Windows 11 laptop, NVIDIA RTX 5060 Laptop GPU, no paid cloud
Institution	_____
Project guide	_____
Date	November 2026

Marker legend: [REV] = section revised from v1.0; [NEW] = section added in v1.1. Values labelled "recommended" are proposals for the team to approve before development starts; they are not measured facts. Appendix A lists every change; Appendix B lists optional suggestions that were not applied.

Table of Contents

1. Product Overview [REV] 3

1.1 Purpose and status of this document . . . 3

1.2 Glossary [REV] . . . 3

2. Product Goals 3

3. Target Users [REV] 3

4. Functional Requirements [REV] 4

4.1 FR-01: Image Upload [REV] . . . 4

4.2 FR-02: Recorded Video Upload [REV] . . . 4

4.3 FR-03: Live Webcam Detection [REV] . . . 5

4.4 FR-04: Detection Classes . . . 5

4.5 FR-05: Model Selection and Fallback [REV] . . . 5

4.6 FR-06: Detection Results [REV] . . . 6

4.7 FR-07: Dashboard . . . 7

4.8 FR-08: Inspection History [REV] . . . 7

4.9 FR-09: Output Management [REV] . . . 7

4.10 FR-10: Error Handling and Job Management [NEW] . . . 8

4.11 Operating Limits [NEW] . . . 8

5. Dataset Requirements [REV] 9

5.1 Acquisition versus training subset [NEW] . . . 9

5.2 Preparation requirements . . . 9

5.3 Splits and leakage prevention [REV] . . . 9

6. Non-Functional Requirements [REV] 10

7. User Workflow [REV] 10

8. Proposed Architecture [REV] 11

9. Out-of-Scope Features [REV] 11

10. Acceptance Criteria [REV] 12

11. Success Metrics [REV] 13

12. Constraints and Risks [REV] 14

13. References 14

Appendix A: Change Log (v1.0 to v1.1) [NEW] 15

Appendix B: Optional Suggestions (not applied) [NEW] 15

1. Product Overview [REV]

RoadVision is a locally operated intelligent road-monitoring application that uses computer vision to detect visible road damage from images, recorded videos and live webcam feeds.

The application uses YOLO26m as its primary detection model and YOLO26n as a lightweight fallback. It provides visual detection results, class-wise summaries and local inspection history. It runs on a single Windows 11 laptop with an NVIDIA RTX 5060 Laptop GPU, with the backend bound to the local machine only.

1.1 Purpose and status of this document

This document specifies what RoadVision must do. It is a requirements specification only: it makes no claim that any feature has been implemented, tested or validated, and it contains no measured results. Implementation status and measured values will be recorded separately in the project report.

1.2 Glossary [REV]

Term	Meaning
RDD2022	Road Damage Detection 2022, the public dataset used for training and evaluation
YOLO26m / YOLO26n	Medium (primary) and nano (fallback) variants of the YOLO26 object-detection model
Confidence score	The model's score for a detection; it is not a guaranteed probability of correctness
Precision / Recall	Fraction of detections that are correct / fraction of labelled damage that is found
mAP@0.5, mAP@0.5:0.95	Mean average precision at an IoU threshold of 0.5, and averaged over thresholds from 0.5 to 0.95
FPS	Frames processed per second
Detection (count)	One bounding box output by the model on one frame. For video and webcam, the same physical damage appears in many frames, so counts are per-frame detections, not unique damage instances (no object tracking is in scope)
Reliable operation	A model is operating reliably if it (a) loads, (b) completes a start-up warm-up inference without error, and (c) for video and webcam, meets the minimum throughput in Section 4.5
Held-out test set	Labelled images never used for training, validation, threshold selection or model selection
Inspection	One processing session: one image, one video job or one webcam session

2. Product Goals

- Identify four selected road-damage categories.
- Provide three input modes: image, recorded video and live webcam.
- Display detection results clearly.
- Provide a simple dashboard for inspections.
- Store inspection history locally.
- Work without paid cloud infrastructure.
- Remain usable when the primary model cannot run reliably.

3. Target Users [REV]

User	Use of the product
Students	Demonstrating the project
Academic evaluators	Assessing the project and its documented results
Users performing preliminary visual road inspections	Obtaining an assistive, preliminary view of visible damage

This is an academic prototype, not a certified road-inspection or engineering assessment system. Detection quality is expected to be lower on inputs that differ from RDD2022 imagery (for example a laptop webcam pointed at a screen or printed photo); the live-webcam mode is a functional demonstration, and its results must not be presented as representative of the measured test-set metrics.

4. Functional Requirements [REV]

ID	Requirement	Summary
FR-01	Image upload	Upload JPG, JPEG or PNG, detect damage, view and download the annotated image
FR-02	Recorded video upload	Process an uploaded video as a background job and produce a browser-playable, downloadable annotated video
FR-03	Live webcam detection	Annotated live feed from a camera on the computer running the backend
FR-04	Detection classes	Four classes mapped consistently across all components
FR-05	Model selection and fallback	YOLO26m primary, tested YOLO26n fallback, never silent; defined fallback triggers
FR-06	Detection results	Per-inspection output, counts and configurable thresholds
FR-07	Dashboard	Simple overview of sessions, detections and active model
FR-08	Inspection history	Local SQLite records of each inspection
FR-09	Output management	Organised, uniquely named annotated files
FR-10	Error handling and job management [NEW]	Defined behaviour for failed uploads, models, cameras and video jobs; cancellation; one job at a time

4.1 FR-01: Image Upload [REV]

The user must be able to upload a supported image file. Supported formats: JPG, JPEG and PNG. The application must:

- Validate file type and size against the limits in Section 4.11. Validation must check that the file actually decodes as an image, not only its extension.
- Display the uploaded image.
- Run road-damage detection.
- Display bounding boxes, labels and confidence scores.
- Show the number of detections by class.
- Allow the annotated image to be viewed and downloaded.

Acceptance criteria: A valid image within limits can be uploaded, processed and displayed with detection results or a clear no-detections message. An invalid, oversized or corrupt file is rejected with a clear message (see AC-02, AC-03).

4.2 FR-02: Recorded Video Upload [REV]

The user must be able to upload a supported recorded video. Supported formats: MP4 and AVI, provided the installed OpenCV/FFmpeg environment can decode them; any other format or codec is rejected with a clear message. The application must:

- Validate the uploaded video (type, size, duration, resolution per Section 4.11, and that it can be opened and decoded) before processing starts.
- Process frames using the selected detection model as a background job, so that the interface stays responsive.
- Show job status and visible progress (frames processed / total frames, or percentage).
- Allow the user to cancel a running job; cancellation must stop processing and remove partial output.
- Generate an annotated output video in a browser-playable format (MP4 with H.264 video). OpenCV's default MP4 writer may produce files that browsers cannot play, so the output must be re-encoded with FFmpeg or

an equivalent where needed.
- Display an annotated preview and provide a download link.
- Show a class-wise detection summary, the number of frames processed, and the number of frames containing at least one detection. Counts are per-frame detections (see Glossary).

Acceptance criteria: A valid video within limits can be processed with visible progress, and an annotated output video can be generated, played in the browser and downloaded (see AC-04, AC-05).

4.3 FR-03: Live Webcam Detection [REV]

The application must support a webcam or USB camera connected to the computer running the backend. The application must:

- Capture camera frames through the chosen camera path (below).
- Display a live annotated feed.
- Show detected classes and confidence scores.
- Provide start and stop controls.
- Display the current active model and the measured live processing FPS.
- Handle camera-unavailable errors clearly (no camera found, camera in use by another application, permission denied, camera disconnected mid-session).
- Run at most one webcam session at a time; end the session automatically at the maximum duration in Section 4.11.
- On stop (or automatic end or failure), save one inspection record containing aggregated counts. Webcam frames are not stored by default.

Camera path decision (mandatory to record before development of FR-03 begins): the project must choose exactly one path and record it in the project report. *Recommended:* backend-side capture with OpenCV (DirectShow backend on Windows), streamed to the React frontend as an annotated MJPEG stream or over a WebSocket. Reason: no per-frame browser upload or encoding, fewer browser-permission variables, and simpler error handling. *Alternative:* browser getUserMedia capture with frames sent to the backend over a WebSocket; this adds encoding overhead and a second source of failure. Since the browser and backend run on the same laptop, both are valid.

Acceptance criteria: The user can start detection, see annotated frames and stop the session without restarting the application. Camera failures produce a clear message and leave the application running (see AC-06, AC-07).

Remote roadside CCTV is outside the project scope.

4.4 FR-04: Detection Classes

The application must recognise the four classes below. The same class names and ordering must be used consistently in the model, backend, database and frontend.

Index	Code	Damage category
0	D00	Longitudinal crack
1	D10	Transverse crack
2	D20	Alligator crack
3	D40	Pothole

Mandatory addition: The class index order above (0 to 3) is the single source of truth. It must be defined once (for example in the dataset YAML and one shared configuration file) and verified by an automated check (AC-08) comparing it with the trained models' class names, the backend, the SQLite schema or constants, and the frontend labels. Annotations in RDD2022 labelled with any other code are excluded from training and the exclusion is recorded (Section 5).

4.5 FR-05: Model Selection and Fallback [REV]

The primary model is YOLO26m and the fallback is YOLO26n. Both checkpoints must be trained or fine-tuned for the four RoadVision classes, and both must use the same class mapping and output format. The application must:

- Attempt to load YOLO26m as the primary model.
- Use YOLO26n if the primary model cannot load or operate reliably (triggers below).
- Display the active model name in the application, together with a visible notice whenever the fallback is active and the reason.
- Log model-loading errors and fallback activation (timestamp, model, reason).
- Store the model actually used in every inspection record.

Condition	Required behaviour
YOLO26m loads and operates reliably	Use YOLO26m; show it as the active model
YOLO26m cannot load or operate reliably	Use the trained and tested YOLO26n; show it as the active model; log the error and the fallback activation
YOLO26n has not been trained and tested	Do not claim the fallback is operational
Neither model is available	Show a clear error (service unavailable); never return misleading results; record failed inspections as failed

Table 5: Model selection behaviour (unchanged from v1.0)

Fallback triggers [NEW] - the application switches from YOLO26m to YOLO26n only on one of the following, and never silently:

#	Trigger	Behaviour
F1	YOLO26m checkpoint missing, corrupt or fails to load at start-up	Start with YOLO26n; log error; show notice
F2	YOLO26m warm-up inference fails at start-up (for example CUDA error or PyTorch/GPU incompatibility)	Start with YOLO26n; log error; show notice
F3	Runtime failure during an image inference (CUDA error, out-of-memory)	Retry once on YOLO26m; if it fails again, switch to YOLO26n, process the request with it, show notice
F4	Runtime failure during a video job	Job restarts once from the beginning on YOLO26n; the UI states the restart; the whole output uses one model. If YOLO26n also fails, the job is marked failed
F5	Live webcam on YOLO26m sustains processed FPS below the minimum live FPS (recommended: 5 FPS) for 10 consecutive seconds on the project hardware	Switch the session to YOLO26n with a visible notice; the inspection record notes both models
F6	Manual selection by the user (optional control)	Use the selected model; show it as active

Rules: (1) YOLO26n counts as an operational fallback only after it has been trained, has passed a smoke test on all three input modes, and its test-set metrics have been recorded; otherwise the interface and report must state that no fallback is available. (2) After a fallback, the application stays on YOLO26n until restart or an explicit user re-check; it does not flip between models automatically. (3) The minimum live FPS value is recommended and is to be confirmed after the first throughput measurement on the project laptop. (4) YOLO26n is expected to be faster and less accurate than YOLO26m; the interface and report must not imply equal accuracy.

4.6 FR-06: Detection Results [REV]

For each inspection, the application must display:

- Annotated image, annotated video or annotated live frame (as applicable to the input mode).
- Damage category.
- Confidence score.
- Total number of detections (per-frame detections; see Glossary).
- Class-wise detection counts.

- Processing status: queued, processing, completed, failed or cancelled.
- Model used and confidence threshold used.

Confidence thresholds must be configurable and documented. *Recommended:* default of 0.25 (the Ultralytics prediction default), adjustable in the interface within 0.10 to 0.90. The default operating threshold used in the final demonstration must be chosen using the validation set, never the test set, and recorded in the report. The NMS IoU threshold used must also be documented. Confidence scores must not be described as guaranteed probabilities of correctness.

4.7 FR-07: Dashboard

The dashboard should provide:

- Total inspection sessions.
- Total detections.
- Class-wise detection counts.
- Recent inspection history.
- Current active model, with a visible fallback notice when applicable.
- Access to image, video and webcam functions.

The dashboard should be simple and readable. Unnecessary charts or features that do not support the demonstration should be avoided. Dashboard totals must be computed from the SQLite records so that they match the history view (AC-13). When there are no records, the dashboard shows an empty state rather than an error.

4.8 FR-08: Inspection History [REV]

SQLite is used for local storage. Each inspection record stores the fields below. Large original videos or images must not be stored inside the database; file paths or managed output references are stored where needed. The user must be able to view previous inspection summaries, and must be able to delete an inspection record together with its output files.

Field	Description
Inspection ID	Unique identifier of the inspection
Date and time	When the inspection was run
Input type	Image, video or webcam
Original filename	Where applicable (sanitised name only)
Model used	YOLO26m or YOLO26n (for a mid-session switch, both are recorded) [REV]
Detection count	Total number of per-frame detections
Class-wise summary	Detections per damage class
Processing status	queued, processing, completed, failed or cancelled [REV]
Confidence threshold [NEW]	Threshold (and NMS IoU) used for the inspection
Processing time [NEW]	Wall-clock processing duration; for video and webcam also frames processed
Output reference [NEW]	Path or managed reference to the annotated output; null if none
Error message [NEW]	Short reason when the status is failed or cancelled; otherwise null

Table 6: Stored inspection fields

4.9 FR-09: Output Management [REV]

The application should organise generated annotated files in a dedicated output directory. Requirements:

- Use unique filenames (for example timestamp plus a random identifier) to prevent overwriting; never use the client-supplied filename as a path.
- Provide access to annotated outputs.

- Handle missing or deleted output files gracefully (history entry remains; the interface shows 'output no longer available').
- Avoid saving unnecessary temporary files indefinitely: temporary files are deleted when a job completes, fails or is cancelled, and leftover temporary files are cleaned at application start-up.
- Uploaded original files are not retained after processing unless the team decides otherwise (recommended, for privacy and disk space); annotated outputs are retained until the user deletes them.
- Check that free disk space is at least twice the input file size before starting a video job; otherwise reject with a clear message.

4.10 FR-10: Error Handling and Job Management [NEW]

The application must handle failures in a defined way and must never present an incomplete or failed result as successful.

Failure	Required behaviour
Unsupported type, oversized, over-duration or corrupt upload	Reject before processing; show a specific message naming the limit or reason; no inspection is processed. A rejected upload may be logged as failed
Image inference error	Apply fallback trigger F3; if all models fail, show an error, record status failed
Video job failure (decode error, model error, disk full)	Apply trigger F4 where relevant; otherwise status failed with a message; delete partial output and temporary files; the application stays usable
Video job exceeds the maximum processing time (recommended: 10 minutes)	Stop the job, status failed with a timeout message, clean up
User cancels a video job	Stop within a few seconds, status cancelled, delete partial output
Camera unavailable, in use, or disconnected mid-session	Clear message; session ends; aggregated results so far are saved with a failed or interrupted note; the application keeps running
Second job requested while one is running	Reject with a 'busy' message (one active video job or webcam session at a time); image requests are processed one at a time
Neither model available	Detection functions disabled with a visible error; history and dashboard remain accessible
SQLite write failure	Show an error; do not lose the annotated output; log the failure

All errors must be logged to a local log file with timestamp and cause. Messages shown to the user must be understandable and must not expose stack traces.

4.11 Operating Limits [NEW]

The values below are **recommended for the team to approve**. They are derived from the intended hardware and the demonstration purpose, not measured; the video limits in particular must be re-checked after the first throughput measurement (Section 11) and adjusted before final acceptance testing. Limits must be configurable constants documented in the README.

Limit	Recommended value	Rationale
Image formats / size	JPG, JPEG, PNG; up to 20 MB	Covers full-resolution phone photographs; larger files add no accuracy because the model resizes input
Image dimensions	Longest side up to 8192 px	Guards against decompression-bomb files; images are resized to the model input size internally
Video formats / size	MP4, AVI; up to 200 MB	Keeps uploads and temporary storage manageable on a laptop
Video duration	Up to 60 seconds	Chosen so that a clip finishes within a few minutes at the throughput expected on the project GPU; confirm after measurement and raise or lower accordingly
Video resolution	Up to 1920 x 1080	Higher resolutions are downscaled by the model and only slow decoding
Video job time limit	10 minutes	Prevents a stuck job from blocking the single job slot

Limit	Recommended value	Rationale
Webcam session length	Automatic stop after 10 minutes	Prevents unbounded memory and aggregation growth during a demonstration
Minimum live FPS (fallback trigger F5)	5 FPS	Below roughly this rate the live feed stops being usable; confirm after measurement
Concurrent jobs	1 video job or webcam session at a time	Single laptop GPU; avoids memory contention

5. Dataset Requirements [REV]

The complete RDD2022 dataset is the acquisition target. Source: https://doi.org/10.6084/m9.figshare.21431547.v1.

5.1 Acquisition versus training subset [NEW]

Acquisition (downloading and preserving the complete dataset) and training (the volume actually used) are separate activities and must be reported separately. The project must not claim that all downloaded images were used for training unless that is verified from the split files. The training subset size is decided from the available hardware and the 7-day schedule, and the selection method (for example stratified sampling by class and country, with a recorded random seed) must be documented. Validation and test sets are not reduced to make training easier; the test set is fixed first and is never sampled from for training.

5.2 Preparation requirements

- Preserve the original downloaded dataset.
- Inspect all available annotations and class distributions.
- Convert Pascal VOC XML annotations to YOLO format where required, mapping to the class order in FR-04 and excluding other labels (recorded).
- Verify bounding-box coordinates (inside image bounds, non-zero area) and class mappings, and review a visual sample of converted labels.
- Keep unlabelled official test images separate from quantitative evaluation.
- Determine actual training volume based on hardware and schedule.
- Record dataset counts, exclusions and split details.

5.3 Splits and leakage prevention [REV]

- Split labelled images at image level into training, validation and testing sets. *Recommended:* approximately 80 / 10 / 10, with the ratio and random seed recorded.
- Prevent duplicate images or leakage across splits: check exact duplicates by file hash, and where the source provides sequential or near-identical frames (for example video-derived images), keep neighbouring frames in the same split.
- Stratify so that every class appears in every split; record per-split class distribution.
- Freeze the split lists in files before training starts; do not regenerate them afterwards.
- The training set trains the model; the validation set may be used for hyper-parameters, checkpoint choice and the confidence threshold; the test set is used only for the final evaluation of the primary and fallback models and is not used to tune anything.
- A script must verify that the three splits are disjoint (AC-16).

The following dataset record will be completed from actual counts during preparation.

Item to record	Value
Images downloaded	To be recorded
Images with usable annotations	To be recorded
Exclusions and reasons (including non-RoadVision labels)	To be recorded

Item to record	Value
Training / validation / test image counts	To be recorded
Class distribution per split	To be recorded
Images actually used for training (subset size and selection method)	To be recorded
Random seed and split-file location	To be recorded
Leakage check result (hash and neighbouring-frame check)	To be recorded

Table 7: Dataset record (to be completed)

6. Non-Functional Requirements [REV]

Category	Requirement
Performance	Image inference should return results without unnecessary delay. Video processing must provide visible progress or processing status. Live-camera inference should remain usable on the available hardware, and the processed FPS must be displayed. Performance must be measured and reported honestly, using the definitions in Section 11.
Reliability	Invalid uploads must produce understandable errors. Model-loading failures must be handled by the rules in FR-05. Camera failures must not crash the application. The system must recover from individual failed inspection sessions and remain usable without a restart (FR-10).
Usability	Clear navigation between image, video, webcam and history. Readable detection labels. Consistent interface layout. Clear loading, empty and error states.
Privacy and security [REV]	Process files locally. Do not upload inspection images or videos to paid cloud services or any external service. Store inspection records locally. The backend must listen on the local machine only (127.0.0.1) unless the team documents a reason otherwise. No external API calls are made during inference.
Platform [NEW]	Windows 11; NVIDIA RTX 5060 Laptop GPU. Frontend: React, TypeScript, Vite and Tailwind CSS. Backend: Python, FastAPI, OpenCV and PyTorch. The PyTorch and CUDA build installed must support this GPU (see Section 12); the backend must also start and run on CPU for fallback and testing, with documented lower speed.
Maintainability [NEW]	Setup and run steps, library versions, limits and thresholds documented in a README. Logging to a local file. Configuration values (limits, thresholds, paths) kept in one place.

7. User Workflow [REV]

Workflow	Steps
Image	1. Open dashboard 2. Select image detection 3. Upload image (invalid file: error shown, return to step 3) 4. Run detection 5. View results, or a no-detections message 6. Download annotated image
Video	1. Open video detection 2. Upload video (invalid or over-limit: error shown) 3. Process video; view progress; cancel if required 4. View annotated preview 5. Download output (On failure: error shown, partial output removed, retry possible)
Live camera	1. Open live detection 2. Start camera (unavailable: error shown) 3. View annotated feed, active model and FPS 4. Stop session (or automatic stop at the limit) 5. Session summary is saved

Workflow	Steps
History	1. Open inspection history 2. Select an inspection 3. View its summary and available output (or 'output no longer available') 4. Optionally delete the inspection

8. Proposed Architecture [REV]

The system consists of the components in Table 10. Figure 1 shows the flow from the user through the frontend and backend to the detection service, the OpenCV and YOLO models, and finally the results, which are saved in SQLite and local output storage.

Component	Responsibility
React frontend (TypeScript, Vite, Tailwind CSS)	User interaction: dashboard, upload forms, results, history, model-status display
FastAPI backend (Python)	Application endpoints, validation, job management (status, progress, cancel), coordination
YOLO26m and YOLO26n inference service (PyTorch)	Loads models, applies fallback rules (FR-05), produces detections
OpenCV	Image, video and camera processing, drawing annotations; FFmpeg-based re-encoding to browser-playable MP4
SQLite	Inspection metadata and summaries
Local storage	Uploaded (temporary) and generated files

Table 10: System components

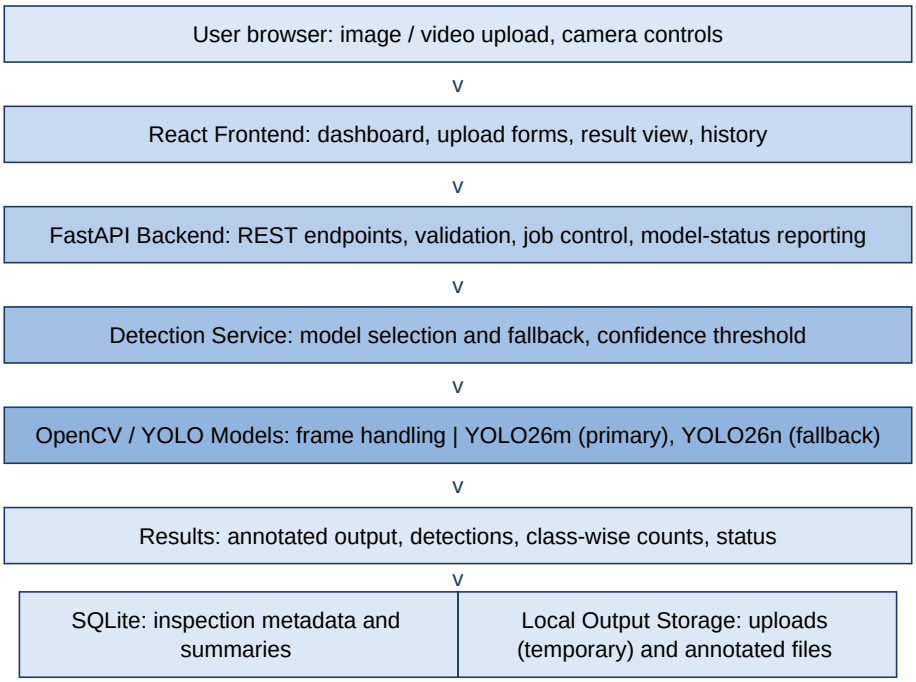


Figure 1: RoadVision system architecture

Architecture decisions (to be recorded when made): (1) camera path - see FR-03; (2) video processing runs as a background job with a status endpoint polled by the frontend; (3) the backend processes one job at a time; (4) both models are loaded through the same inference wrapper so that class mapping and output format are identical.

9. Out-of-Scope Features [REV]

The following are explicitly excluded:

- GPS and mapping.
- Mobile application.
- Remote CCTV.
- Pothole depth estimation.
- Predictive maintenance.
- Automatic road-condition certification.
- Municipal integration.
- Local road-data collection.
- Cloud hosting and paid external APIs.
- Object tracking or de-duplication of the same damage across video frames (counts are per-frame). [NEW]
- User accounts, authentication and multi-user or network access. [NEW]

10. Acceptance Criteria [REV]

The project is considered functionally complete when the criteria below are met. Every functional requirement has at least one criterion. The result column is completed at validation; no result is claimed in this document.

#	Criterion	Verification method	Result
AC-01	The application starts locally on Windows 11 from the documented steps, and the active model is shown. (FR-05, NFR)	Manual start-up from README on the project laptop	To be recorded
AC-02	Image upload and detection work: a valid JPG, JPEG and PNG within limits give an annotated image with boxes, labels, confidence, class counts, or a no-detections message; it can be downloaded. (FR-01, FR-06)	Test with at least one image per format, including one with no damage	To be recorded
AC-03	Invalid images are rejected with a clear message and the app stays usable: wrong type, over the size limit, and a corrupt or renamed file. (FR-01, FR-10)	Negative tests	To be recorded
AC-04	Recorded-video processing works: a valid MP4 (and AVI if supported) within limits shows progress, produces an annotated video that plays in the browser and downloads, with a class-wise summary. (FR-02)	Test with clips within limits	To be recorded
AC-05	Video failures are handled: unsupported codec, over-size or over-duration clip, corrupt file and user cancellation produce a clear message or status, and leave no partial output or temp files. (FR-02, FR-09, FR-10)	Negative tests; inspect output and temp folders	To be recorded
AC-06	Live webcam detection works: start, annotated frames with classes and confidence, FPS shown, stop without restarting; a session record is saved. (FR-03)	Manual test on the laptop camera	To be recorded
AC-07	Camera errors are handled: no camera, camera in use, and camera disconnected give a clear message without crashing. (FR-03, FR-10)	Negative tests	To be recorded
AC-08	All four classes are mapped consistently in dataset config, both models, backend, database and frontend. (FR-04)	Automated check script comparing class lists and order	To be recorded
AC-09	The primary model works: YOLO26m loads at start-up and is shown as active. (FR-05)	Observe model status	To be recorded
AC-10	The tested fallback is activated when required: with the YOLO26m checkpoint removed, and with a simulated runtime failure, YOLO26n becomes active, a notice is shown, the event is logged and the history stores the model used. (FR-05)	Rename checkpoint; inject error; check logs and DB	To be recorded
AC-11	With neither model available, a clear error is shown and no detection results are returned. (FR-05)	Remove both checkpoints	To be recorded

#	Criterion	Verification method	Result
AC-12	Confidence threshold is configurable; changing it changes the displayed detections; the default is documented and the value used is stored with the inspection. (FR-06)	Run one image at two thresholds; check DB	To be recorded
AC-13	Dashboard totals, class counts, recent history and active model are correct and match the SQLite records. (FR-07)	Compare dashboard with a direct DB query	To be recorded
AC-14	Every inspection is saved with all fields in Table 6; records survive an application restart; a deleted output file is handled gracefully; an inspection can be deleted. (FR-08)	Inspect DB; restart; delete a file	To be recorded
AC-15	Annotated outputs can be viewed and downloaded; two uploads with the same filename do not overwrite each other; temporary files are cleaned. (FR-09)	Upload same filename twice; inspect folders	To be recorded
AC-16	Dataset record is complete; training, validation and test splits are disjoint (no hash overlap, neighbouring frames kept together); unlabelled official test images are excluded from evaluation. (Section 5)	Split-check script; review of Table 7	To be recorded
AC-17	Evaluation metrics (precision, recall, mAP@0.5, mAP@0.5:0.95, overall and per class) are calculated for YOLO26m and YOLO26n on the held-out labelled test set. (Section 11)	Evaluation script run once on the frozen test split	To be recorded
AC-18	Privacy: no image or video data leaves the machine during use, and the backend listens on 127.0.0.1 only. (NFR Privacy)	Code review; check listening address; observe network traffic during a run	To be recorded
AC-19	The team can demonstrate the complete workflow end to end, and any incomplete feature is stated as incomplete. (Section 1.1)	Rehearsed demo against this table	To be recorded

Table 11: Acceptance criteria

If a feature is not completed within seven days, it must be identified as incomplete rather than presented as operational.

11. Success Metrics [REV]

Actual measured values will be reported for the metrics below. No target accuracy figures are set, and the system must not be described as production-ready. Metrics must be reported for YOLO26m and YOLO26n separately and on the same held-out test set.

Metric	Measured on	Definition / method	Value
Precision	Held-out labelled test set	Overall and per class, at the documented operating threshold	To be measured
Recall	Held-out labelled test set	Overall and per class, at the documented operating threshold	To be measured
mAP@0.5	Held-out labelled test set	Overall and per class (Ultralytics validation on the frozen test split)	To be measured
mAP@0.5:0.95	Held-out labelled test set	Overall and per class	To be measured
Image inference time	Project hardware	Median and mean over a stated number of runs after a warm-up run, model time only, and end-to-end including upload and drawing	To be measured
Video processing speed (FPS)	Project hardware	Frames processed divided by wall-clock processing time, for a stated clip resolution and length	To be measured
Live-camera performance	Project hardware	Average displayed FPS and approximate end-to-end latency over a stated session length, for each model	To be measured
Functional tests completed successfully	Test checklist	Number of AC-01 to AC-19 passed out of total	To be recorded

Table 12: Success metrics (values to be measured)

Per-class results must be reported because RDD2022 classes are imbalanced; a good overall mAP must not hide a weak class. Test-set results are reported once, after all tuning is complete.

12. Constraints and Risks [REV]

Constraint or risk	Mitigation
Seven-day development period; two-person team	Early integration; clear division of work; incomplete features reported as incomplete. If time runs short, priority order: image, model fallback, history and dashboard, video, live webcam
Limited GPU memory and training time	GPU smoke testing; staged training; fallback-model preparation; do not run training and the application at the same time on the GPU
RTX 5060 (Blackwell) may need a recent PyTorch and CUDA build; an unsupported build fails at run time	Day-1 check that torch.cuda works and a YOLO26 forward pass runs on the GPU; pin the working versions in the README; CPU path as emergency fallback
YOLO26 requires a recent Ultralytics release and pretrained weights; availability or API differences	Day-1 check that both YOLO26m and YOLO26n weights load and train for a few steps; record versions
Large dataset download and preparation time	Staged training on a defined volume; original data preserved
Possible class imbalance	Inspect class distribution; stratified splits; report per-class metrics
Possible annotation conversion errors	Dataset validation of coordinates and class mapping; visual check of converted labels
Split leakage (duplicate or near-identical images)	Hash check and neighbour-frame grouping; frozen split files; single final test evaluation
Browser and local-camera compatibility; Windows camera backends	Early integration testing; clear camera-error handling; use the DirectShow backend; record the chosen camera path
Video codec compatibility, and annotated MP4 not playing in the browser	Test with supported formats early; re-encode output to H.264 MP4 with FFmpeg; clear error for unsupported videos
Webcam inputs differ from RDD2022 imagery (domain shift)	Present live mode as a functional demonstration only; use dash-cam style demo footage; do not quote test-set metrics for it
Live inference too slow on YOLO26m	Fallback trigger F5; measured FPS displayed; report speed and accuracy trade-off honestly

13. References

[1] RDD2022 Dataset: https://doi.org/10.6084/m9.figshare.21431547.v1

[2] RDD2022 Research Paper: https://doi.org/10.1002/gdj3.260

[3] YOLO26 Documentation: https://docs.ultralytics.com/models/yolo26

[4] FastAPI: https://fastapi.tiangolo.com/

[5] OpenCV: https://opencv.org/

[6] React: https://react.dev/

[7] SQLite: https://sqlite.org/

[8] PyTorch: https://pytorch.org/

[9] Vite: https://vite.dev/

[10] Tailwind CSS: https://tailwindcss.com/

[11] TypeScript: https://www.typescriptlang.org/

Appendix A: Change Log (v1.0 to v1.1) [NEW]

M = mandatory correction (a defect, ambiguity or gap that blocks testing or implementation); **R** = recommended value or decision that the team must approve before development. Existing v1.0 requirements were preserved; additions are marked in the body.

Section	Type	Change and reason
Cover, 6	M	Technology stack and platform (TypeScript, Vite, Tailwind, PyTorch, Windows 11, RTX 5060) were absent from the PRD; added as Platform row.
1.2 Glossary	M	Added definitions of detection count, reliable operation, held-out test set and inspection; v1.0 'operate reliably' was untestable.
3	M	Added caution that webcam inputs differ from RDD2022 (domain shift).
4.1 / 4.2	M	Limits referenced; content-based validation; video runs as a background job with progress and cancel; output must be browser-playable H.264 MP4; frames-processed summary.
4.3	M	Camera path ambiguity (browser vs backend capture) resolved into a mandatory recorded decision with a recommendation; error cases listed; session limit; aggregated record saved.
4.4	M	Class index order made the single source of truth; automated consistency check; handling of other RDD2022 labels.
4.5	M	Defined fallback triggers F1 to F6, performance-based fallback, mid-job behaviour, no silent or flip-flop switching, conditions for claiming YOLO26n operational.
4.6	M	Detections are per-frame, not unique damage (no tracking); status values defined; threshold default and range recommended (R); threshold chosen on validation set.
4.8	M	Added fields: threshold, processing time, output reference, error message; both models if switched; status values; delete.
4.9	M	Temp-file cleanup, original-file retention, disk-space check, safe filenames.
4.10 (new)	M	FR-10 defines behaviour for every failure type; error logging.
4.11 (new)	R	Upload, duration, resolution, time and session limits proposed with rationale; to be confirmed after throughput measurement.
5.1 to 5.3	M	Separated complete acquisition from training subset; split ratio, seed, frozen split files, hash and neighbouring-frame leakage checks; test set never used for tuning; extended Table 7.
6	M	Privacy: loopback-only backend, no external calls; maintainability and platform rows.
8	M	Components updated with stack; architecture decisions listed; figure redrawn.
9	M	Added exclusions: object tracking and authentication or network access (no excluded feature from v1.0 was removed).
10	M	Expanded from 10 to 19 criteria so every FR has a testable criterion; added verification method; negative tests, fallback, dashboard, privacy, dataset leakage.
11	M	Metric definitions added; per-class and per-model reporting; latency and FPS defined.
12	M	Added risks: Blackwell GPU and PyTorch build, YOLO26 version availability, split leakage, codec playback, domain shift, priority order if time runs short.

Items to cross-check against the Project Proposal (the proposal was not available for this review, so contradictions could not be identified): number and names of classes; dataset scope and size claims; model names and roles; input modes; stated accuracy or performance targets (this PRD sets none); live-camera approach; hardware and timeline; excluded features.

Appendix B: Optional Suggestions (not applied) [NEW]

- Add a manual model selector (trigger F6) and a 'Re-check YOLO26m' control.
- Optional saving of a snapshot frame when a webcam detection exceeds a set confidence.
- Export an inspection summary to CSV or PDF from the history page.
- Show a lightweight confusion matrix for the test set in the project report only (not in the application).

- Day plan: Day 1 environment, GPU and weights smoke test and dataset conversion; Day 2 start training and build the image pipeline; Day 3 video jobs and history; Day 4 webcam and dashboard; Day 5 evaluation and fallback tests; Day 6 acceptance testing; Day 7 report and demo rehearsal.
- Mark each acceptance criterion Must, Should or Could to guide cuts if the schedule slips.